

Sistemas Operacionais

Sistemas Distribuídos



About me...

- Perito Criminal Federal – Polícia Federal
- Doutorando em Informática pela UFPB
- Mestre em Perícias Forenses pela Universidade de Pernambuco
- Especialista em Criminalística Aplicada a Locais de Crime pela Academia Nacional de Polícia.
- Especialista em Docência do Ensino Superior pela Universidade Federal do Rio de Janeiro.
- Graduado em Ciência da Computação pela Associação Paraibana de Ensino Renovado.
- Graduado em Processamento de Dados pela Associação Paraibana de Ensino Renovado.
- Atuou no Serviço Público como Papiloscopista Policial Federal e também como Policial Rodoviário Federal.
- É Oficial da Reserva de 2ª classe do Exército Brasileiro no posto de Primeiro Tenente da Arma de Cavalaria.
- Trabalhou como Analista de Sistemas na multinacional Xerox do Brasil,
- Foi coordenador de informática na TV Cabo Branco (afiliada da Rede Globo).
- Coautor e revisor dos livros Tratado de Computação Forense, Ciências Forenses - Uma Introdução Às Principais Áreas Da Criminalística, Polícia científica transformando vestígios em evidências à luz da cadeia de custódia.
- Aprovações: PRF 2002, PF 2004, MPU 2010, ABIN 2010, PF 2014
- Contato: gustavopintovilar@gmail.com

Escopo

- Abordar os assuntos mais recorrentes e com fortes tendências para concursos atuais
- Familiarizar o concursando com os tipos de questões mais frequentes.
- Abordar as metodologias de resolução de questões das principais bancas

Sistemas Distribuídos: Definição e Essência

- Definição: Coleção de computadores (nós) *autônomos* que atuam como um *sistema único e coerente* para o usuário final.
- Interconexão: Nodes comunicam-se via rede, trocando mensagens para coordenar ações e compartilhar recursos.
- Autonomia dos Nós: Cada nó possui seu próprio hardware, sistema operacional e software, operando independentemente.
- Visão Unificada: O principal desafio e benefício é apresentar essa complexidade distribuída como uma entidade coesa.

Características Fundamentais de Sistemas Distribuídos

- Transparência: Ocultar a distribuição, localização e heterogeneidade dos componentes
- Concorrência: Múltiplos processos executando simultaneamente em diferentes nós.
- Ausência de Relógio Global: Nós têm seus próprios relógios, que podem divergir.
- Gerenciamento Independente: Cada nó gerencia seus recursos.
- Falhas Independentes: Um nó pode falhar sem afetar os outros imediatamente.

Por Que Usar Sistemas Distribuídos? Vantagens

- Escalabilidade: Capacidade de aumentar a capacidade adicionando mais nós.
- Confiabilidade/Tolerância a Falhas: Falha de um componente não derruba o sistema inteiro.
- Disponibilidade: Serviço acessível por mais tempo devido à redundância.
- Abertura: Facilidade de estender ou modificar o sistema.
- Economia: Usar hardware de commodity mais barato para construir sistemas potentes.
- Desempenho: Processamento paralelo e uso de recursos geograficamente próximos.

Desafios Inerentes dos Sistemas Distribuídos

- Transparência: Difícil de alcançar plenamente.
- Concorrência: Gerenciar acessos simultâneos a recursos compartilhados.
- Falhas de Rede: Comunicação não é garantida; mensagens podem atrasar ou se perder.
- Latência da Rede: Tempo de viagem das mensagens é variável e impacta desempenho.
- Coordenação: Relógios independentes dificultam a ordem global de eventos.
- Segurança: Proteção de dados e comunicação em uma rede heterogênea.
- Complexidade: Maior custo de desenvolvimento, depuração e manutenção.

Arquitetura Distribuída: Princípios e Escolhas

- Definição: Projeto de alto nível de como os componentes de um SD interagem e se organizam.
- Influência: Afeta diretamente a escalabilidade, resiliência, performance e manutenibilidade.
- Decisões Chave:
 - Comunicação: Como os componentes trocam informações.
 - Coordenação: Como as ações são sincronizadas.
 - Estado: Onde e como os dados são armazenados e mantidos consistentes.
 - Falhas: Como o sistema reage a falhas.
- Não Há Solução Única: A melhor arquitetura depende dos requisitos do problema.

Estilos de Arquitetura Distribuída: Uma Visão Geral

- Cliente-Servidor: Tradicional, com papéis bem definidos (servidor fornece, cliente consome).
- Peer-to-Peer (P2P): Descentralizado, nós atuam como clientes e servidores.
- Baseada em Microsserviços: Aplicação dividida em serviços pequenos, independentes e comunicáveis.
- Orientada a Serviços (SOA): Abstração de funcionalidades como serviços reutilizáveis (geralmente monolíticos).
- Orientada a Eventos (Event-Driven): Componentes reagem a eventos gerados por outros componentes.
- Cloud Computing: Abstração e entrega de recursos computacionais via rede.

Modelo Cliente-Servidor: Conceito Base

- Papéis Definidos:
- Servidor: Fornece recursos e serviços; escuta requisições de clientes.
- Cliente: Consome recursos e serviços; inicia requisições ao servidor.
- Comunicação: O cliente solicita, o servidor responde.
- Relação Unidirecional: Clientes normalmente não servem diretamente a outros clientes.
- Centralização Lógica: Servidor como ponto central de gerenciamento de dados/lógica.

Cliente-Servidor: Vantagens e Desvantagens

- Vantagens:

- Gerenciamento Centralizado: Fácil controle de recursos, segurança e atualização.
- Segurança: Mais fácil de implementar e auditar no servidor.
- Compartilhamento de Recursos: Dados e dispositivos centralizados.
- Manutenção: Servidores podem ser atualizados independentemente dos clientes.
- Escalabilidade (Servidor): Possibilidade de escalar o servidor (vertical ou horizontalmente).

- Desvantagens:

- Ponto Único de Falha (SPoF): Se o servidor principal falha, todo o sistema pode parar.
- Gargalo de Desempenho: Servidor pode se tornar um bottleneck sob alta carga.
- Dependência: Clientes dependem do servidor para funcionar.

Variações do Modelo Cliente-Servidor

- Cliente "Thin" (Fino): Maioria da lógica e processamento no servidor; cliente leve (ex: terminais burros, navegador web).
- Cliente "Thick" (Grosso/Rico): Maioria da lógica e processamento no cliente; servidor atua mais como repositório de dados.
- Arquitetura Multi-tier (N Camadas): Separação de responsabilidades em camadas lógicas distintas (apresentação, lógica de negócio, dados).
- Ex: 3 Camadas: Cliente (apresentação) -> Servidor de Aplicação (lógica) -> Servidor de Banco de Dados (dados).
- Proxy/Gateway: Intermediários que encaminham e/ou transformam requisições entre clientes e servidores.

Modelo Peer-to-Peer (P2P): Conceito Base

- Autonomia e Descentralização: Cada nó (peer) é autônomo e não há um servidor central.
- Papéis Fluidos: Cada peer pode atuar como cliente (solicitando recursos) e servidor (fornecendo recursos) simultaneamente.
- Recursos Distribuídos: Dados e serviços estão espalhados entre os peers.
- Comunicação Direta: Peers se comunicam diretamente entre si, sem intermediários.
- Resiliência Inerente: A falha de um ou alguns peers não derruba o sistema.

P2P: Vantagens e Desvantagens

- Vantagens:

- Escalabilidade Massiva: Capacidade de aumentar a capacidade adicionando mais peers à rede.
- Robustez/Tolerância a Falhas: Ausência de SPoF; o sistema continua funcionando mesmo com muitas falhas.
- Custo Reduzido: Não exige infraestrutura de servidor centralizada cara.
- Resistência à Censura: Dificuldade de controlar ou derrubar a rede.
- Performance: Recursos podem ser obtidos de peers mais próximos.

- Desvantagens:

- Gerenciamento e Descoberta: Dificuldade em encontrar recursos ou gerenciar a rede.
- Segurança: Difícil implementar controle de acesso e auditoria; confiança nos peers.
- Confiabilidade: Qualidade dos recursos e disponibilidade dos peers são incertas.
- Consistência: Dificuldade em manter a consistência de dados compartilhados.
- Conformidade: Desafios legais e regulatórios (ex: direitos autorais).

Tipos de Redes P2P

- Não Estruturadas:

- Organização: Sem estrutura fixa; peers se conectam aleatoriamente.
- Descoberta: Flood de requisições pela rede (ex: Gnutella original).
- Vantagem: Fácil de construir, altamente resistente a falhas.
- Desvantagem: Ineficiente para busca, baixa qualidade de resultados.

- Estruturadas:

- Organização: Topologia definida (DHT - Distributed Hash Table).
- Descoberta: Algoritmos específicos garantem busca eficiente (ex: Chord, Kademlia).
- Vantagem: Busca eficiente e garantida.
- Desvantagem: Complexo de construir e manter; menos resistente a falhas massivas.

- Híbridas:

- Organização: Combina elementos P2P e Cliente-Servidor (ex: BitTorrent, Skype).
- Centralização Parcial: Servidores centrais para descoberta de peers ou bootstrapping.
- Vantagem: Melhora a eficiência da busca e confiabilidade com a robustez P2P.
- Desvantagem: Reintroduz SPoF em serviços centralizados.

IPC em Sistemas Distribuídos: Introdução

- Comunicação Essencial: A 'cola' que une os componentes de um sistema distribuído.
- Diferença da IPC Local: Processos em nós distintos, sem memória compartilhada direta.
- Base de Tudo: Sem IPC eficaz, não há sistema distribuído funcional.
- Objetivo: Permitir que processos troquem dados, coordenem ações e compartilhem informações de estado.
- Desafios: A ser detalhado nos próximos slides.

Desafios da Comunicação em Rede

- Latência Inerente: Tempo de viagem das mensagens pela rede é não-zero e variável.
- Falhas de Rede: Mensagens podem ser perdidas, corrompidas ou atrasadas.
- Falhas de Componentes: Servidores podem falhar antes de enviar/receber resposta.
- Serialização/Deserialização: Transformar dados em bytes para transmissão e vice-versa.
- Concorrência: Múltiplas mensagens e acessos simultâneos.
- Ordenação: A ordem de chegada das mensagens pode não ser a ordem de envio.
- Segurança: Interceptação, modificação ou forjamento de mensagens.

Mecanismos Básicos de IPC Distribuída

- Passagem de Mensagens (Sockets):
- Mais baixo nível: Envio/recebimento explícito de blocos de dados (datagramas ou streams).
- Flexível: Grande controle sobre o formato e protocolo.
- Complexo: Exige que o programador gerencie todos os detalhes da comunicação.
- Camadas de Abstração (Middleware):
- RPC (Remote Procedure Call): Simula chamada de função local.
- Mensageria (Message Queues): Comunicação assíncrona via filas.
- APIs Web (REST, gRPC): Estilos arquiteturais e frameworks para comunicação HTTP.
- Objetivo: Simplificar a programação da comunicação distribuída.

RPC (Remote Procedure Call): Conceito e Abstração

- Conceito: Permite que um programa chame uma função ou procedimento em outro computador como se fosse uma função local.
- Abstração Fundamental: Oculta a complexidade da comunicação de rede (endereçamento, transporte, serialização).
- Transparência de Localização: O programador não precisa saber onde o procedimento está sendo executado.
- Objetivo: Facilitar o desenvolvimento de aplicações distribuídas.

RPC: Funcionamento Detalhado - Stubs e Fluxo

- Componentes Chave:

- Cliente Stub: Interface local que empacota a chamada para o servidor.
- Servidor Stub: Interface remota que desempacota a chamada e executa o procedimento.

- Fluxo de Chamada RPC:

- Cliente chama Cliente Stub local.
- Cliente Stub serializa (marshalling) parâmetros.
- Cliente Stub envia mensagem via rede para o servidor.
- Servidor recebe mensagem, Servidor Stub deserializa (unmarshalling).
- Servidor Stub chama o procedimento real no servidor.
- Procedimento executa, retorna resultado para Servidor Stub.
- Servidor Stub serializa (marshalling) o resultado.
- Servidor Stub envia mensagem com resultado via rede para o cliente.
- Cliente recebe mensagem, Cliente Stub deserializa (unmarshalling) o resultado.
- Cliente Stub retorna resultado para o Cliente.

RPC: Vantagens e Limitações

- Vantagens:

- Simplicidade de Programação: Abstrai complexidade de rede, reutiliza paradigmas de programação local.
- Reuso de Código: Procedimentos podem ser compartilhados por múltiplos clientes.
- Distribuição de Processamento: Carga pode ser dividida entre nós.

- Limitações (Transparência Incompleta):

- Semântica de Falha: Falhas de rede ou servidor são diferentes de falhas locais (ex: timeout vs. crash).
- Latência: Chamadas remotas são sempre mais lentas que locais.
- Passagem de Parâmetros: Nem todos os tipos de dados são facilmente serializáveis.
- Acoplamento: Cliente e servidor tendem a ser fortemente acoplados à interface do procedimento.
- Segurança: Desafios de autenticação e autorização em chamadas remotas.

REST

(Representational
State Transfer):
Estilo Arquitetural

- Não é um Protocolo: É um conjunto de princípios arquiteturais para sistemas distribuídos.
- Base: Construído sobre o protocolo HTTP.
- Foco: Manipulação de Recursos nomeados (URLs), não em chamar procedimentos.
- Abordagem Web: Utiliza a filosofia e os mecanismos da World Wide Web.
- Objetivo: Simplicidade, escalabilidade, interoperabilidade entre sistemas heterogêneos.

Componentes Chave de uma API RESTful

- Recursos: Qualquer entidade que pode ser nomeada, endereçada e manipulada (ex: um usuário, um produto).
- URIs (Uniform Resource Identifiers): Identificadores únicos para cada recurso (ex: /api/users/123).
- Métodos HTTP Padrão (Verbos):
 - GET: Recupera o estado de um recurso.
 - POST: Cria um novo recurso ou envia dados para processamento.
 - PUT: Atualiza um recurso existente (substituição completa).
 - PATCH: Atualiza parcialmente um recurso existente.
 - DELETE: Remove um recurso.
- Representações: O formato dos dados trocados (ex: JSON, XML, HTML).
- Statelessness (Sem Estado): Cada requisição do cliente para o servidor deve conter todas as informações necessárias.

REST:

Propriedades e Restrições Arquiteturais

- Cliente-Servidor: Separação de responsabilidades para escalabilidade e portabilidade.
- Sem Estado (Stateless): Servidor não armazena contexto do cliente entre requisições.
- Benefício: Fácil escalabilidade horizontal, resiliência (qualquer servidor pode atender).
- Cacheable: Respostas podem ser armazenadas em cache para melhorar performance.
- Interface Uniforme: Conjunto fixo de operações (métodos HTTP) para todos os recursos.
- Benefício: Simplifica a interação entre componentes.
- Sistema em Camadas: Intermediários (proxies, gateways) podem ser adicionados sem afetar clientes/servidores finais.
- Código Sob Demanda (Opcional): Servidor pode estender funcionalidade do cliente com código executável.

REST:

Vantagens e Desvantagens

- Vantagens:

- Simplicidade e Facilidade de Uso: Baseado em HTTP e princípios web, fácil de entender.
- Escalabilidade: O statelessness e a capacidade de cache favorecem a escalabilidade horizontal.
- Interoperabilidade: Ampla adoção, qualquer plataforma pode consumir.
- Flexibilidade: Permite diferentes formatos de representação (JSON, XML).
- Tolerância a Falhas: Facilita a implementação de resiliência devido ao statelessness.

- Desvantagens:

- Overhead do HTTP: Cabeçalhos podem ser grandes, menos eficiente para pequenas mensagens.
- Não Otimizado para Performance Pura: Latência e throughput podem ser um problema para cenários de alta demanda.
- Sem Tipagem Forte: Dificulta validação e detecção de erros em tempo de compilação.
- Não Otimizado para Real-time/Streaming: Requer mecanismos adicionais (WebSockets).
- Descoberta de API: Menos metadados auto-descritivos que WSDL (SOAP).

gRPC: Evolução do RPC para a Era Cloud e Microserviços

- Contexto: Desenvolvido pelo Google para comunicação interna de alta performance.
- Base: Framework RPC moderno, construído sobre HTTP/2.
- Serialização: Utiliza Protocol Buffers (ProtoBuf) como padrão.
- Foco: Eficiência, performance e tipagem forte para microserviços.
- Objetivo: Superar limitações de RPC tradicionais e REST em cenários de alta demanda.

gRPC: Protocol Buffers - A Linguagem Neutra

- IDL (Interface Definition Language): Usado para definir a estrutura dos dados e os serviços que o gRPC irá expor.
- Serialização Binária: Dados convertidos para um formato binário compacto, em vez de texto (JSON/XML).
- Vantagens:
- Tamanho Reduzido: Mensagens menores (mais rápidas na rede).
- Velocidade: Serialização e desserialização muito mais rápidas.
- Tipagem Forte: Geração de código para diversas linguagens garante consistência de tipos.
- Evolução Simplificada: Compatibilidade backward e forward na evolução de esquemas.
- Desvantagem: Dados não legíveis por humanos (binário).

gRPC: Comunicação e Streaming Avançado

- Base HTTP/2:
 - Multiplexação: Múltiplas requisições/respostas sobre uma única conexão TCP.
 - Compressão de Cabeçalhos: Reduz overhead de requisições.
- Modelos de Intercâmbio:
 - Unary (Request/Response): Chamada RPC tradicional.
 - Server-side Streaming: Cliente envia uma requisição, servidor envia stream de respostas.
 - Client-side Streaming: Cliente envia stream de requisições, servidor envia uma resposta.
 - Bidirectional Streaming: Cliente e servidor enviam streams de dados independentemente.
 - Vantagem: Permite comunicação eficiente em cenários complexos (ex: real-time, IoT).

gRPC: Vantagens e Desvantagens

- Vantagens:
 - Alta Performance: HTTP/2 + Protocol Buffers resultam em alta velocidade e baixo uso de rede.
 - Tipagem Forte: Previne erros de integração em tempo de compilação.
 - Suporte a Streaming: Essencial para aplicações real-time e event-driven.
 - Multi-linguagem: Geração de código para diversas linguagens.
 - Eficiência de Bateria/CPU: Menor uso de recursos em dispositivos móveis.
- Desvantagens:
 - Curva de Aprendizado: Mais complexo que REST para iniciar.
 - Legibilidade: Mensagens binárias não são facilmente inspecionáveis por humanos.
 - Ecossistema: Menos maduro que REST (ferramentas, browsers).
 - Proxies HTTP: Nem todos os proxies suportam HTTP/2 e gRPC.
 - Firewalls: Pode exigir configuração específica para port-forwarding.

Sincronização em Sistemas Distribuídos: O Problema

- Desafio Central: Como coordenar ações e garantir a ordem correta de eventos em um sistema sem um relógio global único.
- Assincronia Inerente: Comunicação via rede é não-determinística, mensagens podem atrasar ou chegar fora de ordem.
- Consistência de Estado: Múltiplos nós precisam concordar sobre o estado do sistema ou de dados compartilhados.
- Acesso Concorrente: Evitar que múltiplos nós tentem modificar o mesmo recurso simultaneamente.
- Problemas: Estados inconsistentes, dados corrompidos, decisões contraditórias.

Tipos de Sincronização Distribuída

- Sincronização de Tempo:
 - Relógios Físicos: Ajustar relógios para que se aproximem do tempo real (ex: NTP).
 - Relógios Lógicos: Estabelecer uma ordem causal entre eventos, sem depender de relógios físicos (ex: Lamport, Vetoriais).
- Sincronização de Exclusão Mútua:
 - Garantir que apenas um processo (ou nó) acesse um recurso compartilhado por vez.
 - Essencial para manter a consistência de dados em ambientes distribuídos.
- Sincronização de Dados (Consistência):
 - Garantir que as réplicas de dados em diferentes nós permaneçam coerentes entre si.

Desafios Adicionais na Sincronização Distribuída

- Paralelismo Genuíno: Operações realmente simultâneas, não apenas intercaladas.
- Desconhecimento do Estado Global: Nenhum nó tem uma visão completa e atual do estado de todos os outros nós.
- Liveness (Vivacidade): Garantir que a sincronização não cause deadlock ou starvation.
- Segurança: Prevenir que um nó malicioso manipule a sincronização.
- Otimização: Sincronização tem custo; otimizar para desempenho e escalabilidade.

Relógios Físicos vs. Relógios Lógicos

- Relógios Físicos:
 - Objetivo: Medir o tempo real absoluto (ex: UTC).
 - Implementação: Hardware (cristal de quartzo); software ajusta (ex: NTP).
 - Problema: Divergência de relógios (drift); não garantem ordem causal precisa.
- Relógios Lógicos:
 - Objetivo: Estabelecer uma ordem causal relativa entre eventos.
 - Implementação: Contadores incrementados por regras específicas.
 - Benefício: Resolvem o problema da ordenação de eventos em sistemas distribuídos sem depender da precisão dos relógios físicos.

Algoritmo de Lamport: A Relação "Aconteceu Antes"

- Proposto por: Leslie Lamport (1978).
- Ideia Central: Definir uma relação de ordem parcial "aconteceu antes" ($a \rightarrow b$) entre eventos em um SD.
- Regras:
 - Se a e b são eventos no mesmo processo e a ocorre antes de b , então $a \rightarrow b$.
 - Se a é o evento de envio de uma mensagem m por um processo e b é o evento de recebimento de m por outro processo, então $a \rightarrow b$.
 - Se $a \rightarrow b$ e $b \rightarrow c$, então $a \rightarrow c$ (transitividade).
- Implementação: Cada processo mantém um contador (relógio lógico).
- Incrementa seu contador antes de cada evento interno.
- Ao enviar uma mensagem, inclui seu contador atual.
- Ao receber uma mensagem, atualiza seu contador para $\max(\text{local_counter}, \text{message_counter}) + 1$.
- Limitação: Não captura concorrência (se a e b não são $a \rightarrow b$ nem $b \rightarrow a$, não significa que são concorrentes).

Relógios Vetoriais: Capturando a Concorrência

- Objetivo: Superar a limitação de Lamport, capturando não apenas a ordem causal, mas também a concorrência entre eventos.
- Implementação: Cada processo mantém um vetor de contadores (um contador para cada processo no sistema).
- Regras:
 - Antes de cada evento interno i , o processo P_i incrementa $V_i[i]$.
 - Ao enviar uma mensagem, o processo P_i inclui seu vetor V_i completo.
 - Ao receber uma mensagem m com vetor V_m , o processo P_j atualiza seu vetor V_j para $\max(V_j[k], V_m[k])$ para cada k , e então incrementa $V_j[j]$.
- Comparação: Permite determinar se $a \rightarrow b$, $b \rightarrow a$ ou $a \parallel b$ (concorrentes).
- Vantagem: Mais preciso para detecção de causalidade e concorrência.
- Desvantagem: Overhead maior (tamanho do vetor cresce com o número de processos).

Comparativo: Relógios Físicos, Lamport e Vetoriais

Característica	Relógios Físicos (ex: NTP)	Relógio de Lamport	Relógios Vetoriais
Objetivo	Aproximar o tempo real (UTC)	Definir ordem causal parcial (aconteceu antes)	Definir ordem causal total e concorrência
Base	Hardware (oscilador) + Ajuste Software	Contadores escalares de Software	Vetores de contadores de Software
Precisão	Pode divergir (drift), não garante causalidade	Garante causalidade se $a \rightarrow b$, mas não concorrência	Garante causalidade e detecta concorrência
Custo	Hardware e Software (NTP)	Baixo (escalar)	Médio (vetor cresce com nº de processos)
Complexidade	Média (sincronização de hardware)	Baixa (regras simples)	Média (gerenciar e comparar vetores)

Consistência: Um Desafio Fundamental

- Definição: Em um sistema distribuído, consistência refere-se à propriedade de que todos os nós de dados (réplicas) concordam sobre o valor de um dado a qualquer momento.
- Ideal: Todos os leitores veem sempre o dado mais recente e correto.
- Realidade: Em sistemas distribuídos, é difícil manter consistência forte com alta disponibilidade.
- Problema: Operações concorrentes (leituras/escritas) em diferentes nós podem levar a estados inconsistentes.
- Para Concursos: Um dos pilares do design de sistemas distribuídos, frequentemente contrastado com Disponibilidade.

Tipos de Consistência

(I): Forte e Fraca

- Consistência Forte (Strong Consistency):
 - Propriedade: Uma vez que uma escrita é completada, todas as leituras subsequentes verão o dado atualizado.
 - Modelo: Linearizável, Sequencial (ACID).
 - Vantagem: Dados sempre corretos e atualizados.
 - Desvantagem: Alta latência, baixa disponibilidade em caso de falha de rede/nó (Teorema CAP).
 - Uso: Sistemas financeiros, transações críticas.
- Consistência Fraca (Weak Consistency):
 - Propriedade: Após uma escrita, leituras subsequentes podem ou não ver o dado atualizado.
 - Vantagem: Alta disponibilidade e desempenho.
 - Desvantagem: Dados podem estar desatualizados temporariamente.
 - Uso: Caches, contadores de "curtidas" em redes sociais.

Tipos de Consistência (II): Eventual, Causal e Outros

- Consistência Eventual (Eventual Consistency):
 - Propriedade: Se nenhuma nova escrita ocorrer em um dado por tempo suficiente, eventualmente todas as réplicas convergirão para o mesmo valor.
 - Modelo: BASE (Basically Available, Soft state, Eventually consistent).
 - Uso: Sistemas que podem tolerar desatualização temporária (DNS, serviços de e-mail, alguns NoSQL).
- Consistência Causal (Causal Consistency):
 - Propriedade: Se um processo 'a' causou 'b', então todos os nós que veem 'b' devem ver 'a' primeiro.
 - Foco: Preservar a ordem causal dos eventos.
- Outros Modelos (Exemplos): Read-your-writes, Monotonic Read/Write.
 - Trade-off: A complexidade da garantia de consistência versus desempenho, disponibilidade e latência.

Consenso: Acordo em um Mundo Distribuído

- Definição: O problema de fazer com que múltiplos nós em um sistema distribuído concordem sobre um único valor ou decisão.
- Contexto: Mesmo na presença de falhas (nós parando, mensagens perdidas).
- Objetivo: Garantir a atomicidade de decisões em ambientes distribuídos.
- Problemas Solucionados: Eleição de líder, confirmação de transações, replicação consistente.
- Desafio: Alcançar consenso de forma eficiente e tolerante a falhas.

Propriedades Essenciais dos Algoritmos de Consenso

- Validade: Se um valor é decidido, ele deve ter sido proposto por um nó.
- Concordância (Agreement): Se um nó decide um valor 'v', então nenhum outro nó decide um valor diferente de 'v'.
- Terminação (Termination): Se o nó líder não falhar e a rede for confiável, todos os nós eventualmente decidem um valor.
- Tolerância a Falhas: Devem funcionar mesmo com a falha de uma minoria de nós.
- Livreness: A decisão eventualmente será tomada.
- Para Concursos: Conhecer essas propriedades é chave para entender a robustez dos algoritmos.

Paxos: O Primeiro Algoritmo de Consenso (Líder Eleito)

- Criador: Leslie Lamport (1989).
- Problema Resolvido: Alcançar consenso em um ambiente distribuído e assíncrono com falhas de nós e perda de mensagens.
- Papéis: Proposer (propõe um valor), Acceptor (vota), Learner (aprende o valor decidido).
- Fases:
 - Prepare: Proposer busca um quórum de Acceptors.
 - Accept: Proposer envia seu valor para o quórum de Acceptors.
- Característica: Complexo de entender e implementar.

Paxos: Desafios e Variações

- Dificuldade: Conhecido pela dificuldade de compreensão e implementação correta.
- Liderança Flexível: Não exige um líder fixo, mas a eleição de um pode simplificar.
- Variações:
 - Multi-Paxos: Permite que múltiplas instâncias de consenso sejam executadas para acordar uma sequência de valores.
 - Fast Paxos: Otimização para reduzir o número de rodadas de comunicação.
- Uso: Embora complexo, é a base para sistemas como Google Chubby, Apache ZooKeeper (em sua essência).

Raft: Consenso para Entender e Implementar

- Objetivo: Ser um algoritmo de consenso mais compreensível e implementável que Paxos.
- Foco: Facilidade de entendimento e forte garantia de segurança (nunca retorna valores incorretos).
- Conceito Principal: Consenso baseado na replicação de um log.
- Papéis: Leader (líder), Follower (seguidor), Candidate (candidato).
- Estados:
 - Follower: Passivo, recebe requisições do Leader.
 - Candidate: Ativo, tenta se tornar Leader.
 - Leader: Ativo, gerencia o log de replicação.

Raft: Eleição de Líder e Replicação de Log

- Eleição de Líder:
 - Followers têm um timeout aleatório para eleição.
 - Quando um Follower não ouve do Leader, torna-se Candidate e solicita votos.
 - Se um Candidate obtém votos da maioria, torna-se Leader.
- Replicação de Log:
 - Todas as alterações de estado são enviadas ao Leader.
 - Leader anexa a alteração ao seu log e envia para os Followers.
 - Followers confirmam o recebimento.
 - Quando a maioria dos Followers replicou, o Leader commita a entrada e responde ao cliente.
 - Vantagem: Forte consistência (linearizabilidade) com alta disponibilidade em caso de falha de até $(N-1)/2$ nós.

Raft: Propriedades e Uso Prático

- Consistência: Garante linearizabilidade (a forma mais forte de consistência) para operações comitadas.
- Tolerância a Falhas: Funciona corretamente desde que a maioria dos nós esteja operacional.
- Casos de Uso:
 - Sistemas de coordenação distribuída (ex: etcd, HashiCorp Consul).
 - Serviços de metadados em cluster (Kubernetes).
 - Replication de logs de banco de dados.
- Popularidade: Preferido por muitos projetos e engenheiros devido à sua clareza.

Replicação: Copiando Dados para Resiliência e Desempenho

- Definição: Manter múltiplas cópias de um mesmo dado (ou serviço) em diferentes nós de um sistema distribuído.
- Objetivos Principais:
 - Tolerância a Falhas: Se um nó falha, outras cópias estão disponíveis.
 - Disponibilidade: Aumenta a probabilidade de o serviço estar sempre acessível.
 - Desempenho (Escalabilidade de Leitura): Permite distribuir leituras entre as réplicas.
- Desafio: Manter a consistência entre as réplicas (problema da consistência).

Tipos de Replicação (I): Ativa vs. Passiva

- Replicação Ativa (Active Replication):
 - Funcionamento: Todas as réplicas processam todas as requisições de forma independente e concordam na ordem das operações (requer consenso).
 - Vantagem: Rápida, alta tolerância a falhas (nós podem falhar a qualquer momento sem interrupção).
 - Desvantagem: Mais complexa de implementar (necessita de um algoritmo de consenso).
 - Uso: Sistemas com requisitos de latência muito baixos, alto throughput.
- Replicação Passiva (Passive Replication):
 - Funcionamento: Existe um nó primário (Leader) que processa todas as requisições e replica o estado para os nós secundários (Standby/Followers).
 - Vantagem: Mais simples de implementar, menos overhead de consenso.
 - Desvantagem: Pode haver latência durante a troca de líder em caso de falha do primário.
 - Uso: Muitos bancos de dados replicados.

Tipos de Replicação (II): Síncrona vs. Assíncrona

- Replicação Síncrona:
 - Funcionamento: A operação de escrita no primário só é considerada completa quando confirmada por um número predefinido de réplicas.
 - Garantia: Forte consistência (sem perda de dados em caso de falha do primário).
 - Desvantagem: Maior latência na escrita, pode reduzir a disponibilidade se muitas réplicas falharem.
 - Uso: Transações financeiras, dados críticos.
- Replicação Assíncrona:
 - Funcionamento: O primário conclui a escrita e responde ao cliente antes que a replicação para as outras réplicas esteja completa.
 - Garantia: Consistência eventual, possível perda de dados recentes em caso de falha do primário antes da replicação.
 - Vantagem: Baixa latência na escrita, alta disponibilidade.
 - Uso: Sistemas com alto throughput, menos críticas a perda de dados recentes (redes sociais, logs)

Particionamento de Dados: Dividir para Conquistar

- Definição: Dividir um grande conjunto de dados em subconjuntos menores e independentes (partições) que podem ser armazenados e gerenciados em diferentes nós.
- Sinônimos: Sharding, Fragmentação.
- Objetivos Principais:
 - Escalabilidade Horizontal: Distribuir a carga de dados e requisições por mais máquinas.
 - Desempenho: Reduzir o volume de dados a ser pesquisado/processado por cada nó.
 - Disponibilidade: Falha de uma partição afeta apenas parte do sistema.
- Desafio: Distribuição uniforme dos dados, coordenação entre partições.

Estratégias de Particionamento

- Particionamento por Chave (Key-Based/Hash Partitioning):
 - Funcionamento: Usa uma função de hash sobre a chave de um registro para determinar a partição.
 - Vantagem: Distribuição uniforme de dados, simples de implementar.
 - Desvantagem: Não otimizado para consultas por faixa (range queries).
- Particionamento por Faixa (Range Partitioning):
 - Funcionamento: Divide os dados com base em faixas de valores da chave (ex: A-M em uma partição, N-Z em outra).
 - Vantagem: Otimizado para consultas por faixa.
 - Desvantagem: Possibilidade de desequilíbrio (hot spots) se os dados não forem uniformemente distribuídos.
- Particionamento por Lista/Diretório:
 - Funcionamento: Mapeia chaves para partições usando uma lista explícita ou um serviço de diretório.
 - Vantagem: Flexível.
 - Desvantagem: Requer gerenciamento externo.

Sharding: O Particionamento em Ação

- Definição: Uma técnica específica de particionamento de banco de dados onde cada partição (shard) é um banco de dados independente e completo.
- Shard: Um subconjunto do banco de dados completo, contendo seus próprios dados, índice e esquema.
- Vantagens:
 - Escalabilidade Extrema: Suporta volumes massivos de dados e tráfego.
 - Isolamento de Falhas: Falha de um shard afeta apenas a porção de dados que ele armazena.
- Desafios:
 - Rebalanceamento: Distribuir dados entre shards existentes ou novos.
 - Consultas Distribuídas: Consultas que exigem dados de múltiplos shards.
 - Transações Distribuídas: Manter ACID em operações que afetam múltiplos shards.

Sharding: Chave de Particionamento (Shard Key)

- Shard Key (Chave de Particionamento): Coluna ou conjunto de colunas usada para determinar qual shard armazenará um dado específico.
- Importância: A escolha da shard key é a decisão mais crítica no design de um sistema shardeado.
- Características de uma Boa Shard Key:
 - Alta Cardinalidade: Muitos valores únicos para evitar hot spots.
 - Distribuição Uniforme: Espalha os dados de forma equilibrada entre os shards.
 - Suporte a Consultas Comuns: Permite que a maioria das consultas acesse apenas um shard.
- Exemplos: ID do usuário, ID da transação, ID do cliente (com hash).

Tolerância a Falhas: Resiliência em Sistemas Distribuídos

- Definição: A capacidade de um sistema continuar operando corretamente (mesmo que com degradação de performance) na presença de falhas de um ou mais componentes.
- Contraste: Não é o mesmo que prevenção de falhas; é a capacidade de lidar com elas.
- Falhas Comuns:
 - Falhas de hardware (disco, CPU, memória).
 - Falhas de software (bugs, travamentos de processo).
 - Falhas de rede (perda de pacotes, latência, partições).
- Objetivo: Minimizar o impacto de falhas no serviço total.
- Para Concursos: Um requisito crucial para sistemas de missão crítica.

Estratégias para Tolerância a Falhas

- Detecção de Falhas: Identificar que um componente falhou (heartbeats, timeouts).
- Replicação: Manter múltiplas cópias de dados/serviços (já discutido).
- Redundância: Ter componentes sobressalentes para assumir em caso de falha.
- Checkpointing e Rollback: Salvar o estado do sistema periodicamente e restaurar para um estado conhecido.
- Timeouts e Retentativas: Tentar novamente uma operação em caso de falha temporária.
- Isolamento de Falhas: Conter o impacto de uma falha a uma parte do sistema (circuit breakers).
- Design para Falha: Assumir que falhas ocorrerão e projetar o sistema com isso em mente.

Disponibilidade: Serviço Sempre Acessível

- Definição: A probabilidade de um sistema estar operacional e acessível quando necessário.
- Métrica: Medida em percentual de tempo de atividade (ex: "cinco noves" = 99.999% de disponibilidade).
- Objetivo: Minimizar o tempo de inatividade (downtime).
- Fatores que Afetam: Falhas de hardware/software, problemas de rede, manutenção planejada, desastres.
- Relação com Tolerância a Falhas: Sistemas tolerantes a falhas tendem a ser mais disponíveis.

Alta Disponibilidade (HA): Objetivos e Implementação

- Definição: Um conjunto de práticas, tecnologias e estratégias para garantir que um sistema ou serviço opere continuamente por um período prolongado.
- Objetivo: Minimizar o tempo de inatividade para um nível aceitável ou zero.
- Componentes Chave:
- Redundância: Duplicar componentes críticos (servidores, redes, fontes de energia).
- Failover Automático: Detecção rápida de falha e transferência automática para um componente redundante.
- Monitoramento: Vigilância constante da saúde do sistema.
- Manutenção sem Downtime: Atualizações e reparos sem parar o serviço.

Teorema CAP: O Trilema da Consistência

- Criador: Eric Brewer (2000).
- Enunciado: Em um sistema distribuído, é impossível garantir simultaneamente:
- Consistência (todos os nós veem o mesmo dado).
- Availability (o sistema sempre responde a uma requisição).
- Partition Tolerance (o sistema continua operando mesmo com falhas de comunicação entre nós).
- Escolha: Você pode ter no máximo duas dessas três propriedades a qualquer momento.
- Implicância: A escolha dos dois prioritários define a arquitetura do sistema.

Teorema CAP: CP (Consistência e Tolerância a Partição)

- Prioridade: Consistência e Tolerância a Partição.
- Comportamento:
 - Em caso de partição de rede, o sistema prefere parar de aceitar requisições em um lado da partição para garantir que não haja inconsistências.
 - Ou seja, sacrifica a Disponibilidade.
- Características:
 - Garantia de que os dados são sempre corretos.
 - Pode ficar temporariamente indisponível.
- Uso:
- Bancos de dados transacionais (ex: RDBMS tradicionais).
- Sistemas que exigem forte integridade de dados (sistemas financeiros, bancos de dados que implementam ACID).

Teorema CAP: AP (Disponibilidade e Tolerância a Partição)

- Prioridade: Disponibilidade e Tolerância a Partição.
- Comportamento:
 - Em caso de partição de rede, o sistema continua aceitando requisições em todos os lados da partição.
 - Ou seja, sacrifica a Consistência.
- Características:
 - Sempre acessível (pode fornecer dados desatualizados).
 - Consistência eventual.
- Uso:
 - Serviços de redes sociais, e-commerce (carrinhos de compra), DNS.
 - Bancos de dados NoSQL (ex: Cassandra, DynamoDB).

Teorema CAP: Considerações Práticas

- Não é um "tudo ou nada": Na prática, a escolha é um espectro, não binária. Muitos sistemas buscam um equilíbrio.
- Disponibilidade vs. Performance: Consistência forte geralmente implica menor performance.
- Consistência Eventual é Aceitável: Para muitos casos de uso, inconsistência temporária é tolerável.
- Falha de Rede é a Realidade: A Partição é inevitável em SDs, tornando a escolha C ou A obrigatória.
- Impacto no Design: Define profundamente a arquitetura de armazenamento e replicação de dados.

Redundância: Multiplicando para Garantir

- Definição: Duplicação de componentes, dados ou funções em um sistema para aumentar a tolerância a falhas.
- Objetivo: Prover caminhos alternativos de operação ou armazenamento em caso de falha.
- Tipos de Redundância:
 - Hardware: Múltiplos servidores, fontes de energia, discos (RAID), placas de rede.
 - Software: Múltiplas instâncias de uma aplicação, bancos de dados replicados.
 - Rede: Múltiplos links, roteadores, switches.
- Benefícios: Aumenta a disponibilidade e a confiabilidade.

Redundância Hot vs. Cold

- Redundância Hot (Ativa/Ativa):
 - Funcionamento: Todos os componentes redundantes estão ativos e processando requisições simultaneamente.
 - Vantagem: Failover instantâneo, sem perda de sessão, aproveitamento total dos recursos.
 - Desvantagem: Mais complexo de configurar e sincronizar.
 - Uso: Balanceamento de carga, replicação ativa.
- Redundância Cold (Passiva/Standby):
 - Funcionamento: Componentes redundantes estão inativos ou em modo de espera, ativados apenas em caso de falha do primário.
 - Vantagem: Mais simples de implementar, menor consumo de recursos (se o standby for 'cold').
 - Desvantagem: Tempo de inatividade durante o failover, potencial perda de dados recentes.
 - Uso: Replicas passivas, backups.
- Redundância Warm: Intermediária entre hot e cold (standby ativo, mas não processando todas as requisições).

Escalabilidade: Crescendo com a Demanda

- Definição: A capacidade de um sistema aumentar seu desempenho ou throughput sob uma carga crescente, sem degradação significativa.
- Objetivo: Adaptar-se ao crescimento da demanda de usuários, dados ou requisições.
- Importância: Manter a qualidade do serviço à medida que a base de usuários e o volume de dados crescem.
- Métrica: Como o desempenho muda quando mais recursos são adicionados.

Escalabilidade Horizontal (Scale Out)

- Definição: Aumentar a capacidade adicionando mais servidores (nós) a um cluster existente.
- Estratégia: Distribuir a carga de trabalho entre um número maior de máquinas, muitas vezes máquinas commodity (mais baratas).
- Vantagens:
 - Custo-benefício: Geralmente mais barato que vertical.
 - Tolerância a Falhas: A falha de um nó não derruba o sistema.
 - Flexibilidade: Fácil adicionar/remover nós.
- Desvantagens:
 - Complexidade: Requer gerenciamento de distribuição, consenso, consistência.
 - Latência: Comunicação entre nós.
- Uso: Web servers, bancos de dados NoSQL, microserviços.

Escalabilidade Vertical (Scale Up)

- Definição: Aumentar a capacidade aprimorando os recursos de um único servidor (CPU, RAM, armazenamento).
- Estratégia: Tornar a máquina existente mais potente.
- Vantagens:
 - Simplicidade: Sem complexidade de sistemas distribuídos.
 - Desempenho: Aproveita ao máximo a proximidade dos recursos.
- Desvantagens:
 - Custo: Máquinas superpotentes são exponencialmente mais caras.
 - Limite Físico: Sempre haverá um limite para a potência de uma única máquina.
 - Ponto Único de Falha: Falha de uma máquina única derruba o serviço.
- Uso: Bancos de dados relacionais (em menor escala), aplicações legadas.

Elasticidade: Reagindo à Demanda Dinâmica

- Definição: A capacidade de um sistema de escalar horizontalmente (adicionar ou remover recursos) automaticamente e dinamicamente em resposta a mudanças na carga de trabalho.
- Objetivo: Otimizar o uso de recursos e custos, mantendo o desempenho.
- Escalabilidade vs. Elasticidade:
 - Escalabilidade: Capacidade de lidar com aumento de carga.
 - Elasticidade: Capacidade de escalar automaticamente *para cima e para baixo* conforme a necessidade.
- Componentes Chave:
 - Monitoramento de métricas (CPU, memória, requisições).
 - Políticas de autoescalonamento.
 - Infraestrutura que suporte criação/destruição rápida de recursos (nuvem, contêineres).

Elasticidade: Cenários e Benefícios

- **Cenários de Uso:**

- Picos de tráfego imprevisíveis (campanhas de marketing, eventos).
- Cargas de trabalho sazonais (Natal, Black Friday).
- Ambientes de desenvolvimento e teste (provisionamento on-demand).

- **Benefícios:**

- Otimização de Custos: Paga apenas pelos recursos que realmente usa.
- Desempenho Otimizado: Mantém a performance mesmo sob picos de carga.
- Agilidade: Adaptação rápida a mudanças na demanda.
- Redução de Over-provisioning: Evita manter recursos ociosos.
- Tecnologias Habilitadoras: Virtualização, contêineres, orquestradores (Kubernetes).

Clustering: Agrupando para Colaborar

- Definição: Um grupo de servidores (nós) interconectados que trabalham juntos como uma única entidade lógica.
- Objetivos:
 - Alta Disponibilidade (HA): Fornecer failover automático em caso de falha de um nó.
 - Balanceamento de Carga: Distribuir requisições entre os nós do cluster.
 - Escalabilidade: Aumentar a capacidade total ao adicionar mais nós.
- Tipos de Clustering:
 - Failover Clusters: Foco em HA, um nó ativo e outros em standby.
 - Load Balancing Clusters: Foco em desempenho e escalabilidade, todos os nós ativos.
 - High-Performance Computing (HPC) Clusters: Foco em processamento paralelo intenso.

Vantagens e Desvantagens do Clustering

- Vantagens:

- Maior Disponibilidade: Redução do tempo de inatividade.
- Melhor Desempenho: Distribuição de carga, processamento paralelo.
- Escalabilidade Flexível: Fácil adicionar/remover nós.
- Eficiência de Recursos: Otimiza o uso de hardware.

- Desvantagens:

- Complexidade: Gerenciamento, configuração, monitoramento de múltiplos nós.
- Custo: Hardware adicional, software de cluster, manutenção.
- Sincronização: Manter o estado consistente entre os nós.
- Single Point of Failure (SPOF) Remanescente: A coordenação do cluster pode se tornar um SPOF se não for bem projetada.

Virtualização em Ambientes Distribuídos

- Definição: Criação de uma versão virtual de algo (sistema operacional, servidor, recurso de armazenamento ou rede).
- Benefícios em SD:
 - Isolamento: VMs isolam aplicações e SOs uns dos outros.
 - Portabilidade: Migrar VMs entre hosts físicos facilmente.
 - Utilização de Recursos: Consolidação de servidores, melhor uso do hardware.
 - Elasticidade: Provisionamento rápido de novas VMs.
- Tecnologias: Hypervisors (VMware ESXi, KVM, Hyper-V).

Tipos de Virtualização (I)

- Virtualização de Servidor:
 - Definição: Criação de múltiplas VMs em um único servidor físico.
 - Software: Hypervisor (Type 1 ou Type 2).
 - Vantagens: Consolidação de hardware, isolamento de aplicações.
 - Uso: Data centers, nuvem privada, hospedar múltiplos serviços em um servidor.
- Virtualização de Rede (SDN - Software-Defined Networking):
 - Definição: Abstração da infraestrutura de rede, permitindo gerenciar e programar a rede via software.
 - Vantagens: Flexibilidade, provisionamento rápido de redes virtuais, otimização de tráfego.
 - Uso: Redes de data center, nuvem pública, microsserviços.
- Virtualização de Armazenamento:
 - Definição: Agrega múltiplos dispositivos de armazenamento físico em um único pool lógico.
 - Vantagens: Gerenciamento centralizado, otimização de espaço, tiering de dados.

Tipos de Virtualização (II)

- Virtualização de Desktop:
 - Definição: Fornecimento de ambientes de desktop (SO e aplicações) de forma centralizada a usuários remotos (VDI - Virtual Desktop Infrastructure).
 - Vantagens: Gerenciamento centralizado, segurança, acesso remoto.
 - Uso: Call centers, ambientes corporativos seguros.
- Virtualização de Aplicações:
 - Definição: Executar aplicações em um ambiente isolado do SO subjacente (sem virtualizar o SO completo).
 - Vantagens: Compatibilidade, isolamento de aplicações, portabilidade.
 - Uso: Softwares legados, apps em quiosques, contêineres (forma de virtualização de aplicação).
- Virtualização de Dados:
 - Definição: Criação de uma camada de abstração sobre fontes de dados heterogêneas, apresentando-as como uma única fonte.
- Vantagens: Integração de dados, agilidade, segurança.

Contêineres em Ambientes Distribuídos: Leveza e Agilidade

- Definição: Pacotes de software que incluem o código da aplicação, bibliotecas, dependências e configurações necessárias para rodar.
- Mecanismo: Compartilham o kernel do sistema operacional host, mas isolam processos, sistemas de arquivos e redes.
- Diferença da VM: Mais leves, iniciam mais rapidamente, consomem menos recursos.
- Tecnologia Líder: Docker.
- Benefícios em SD:
 - Consistência de Ambiente: "Funciona na minha máquina" = "Funciona em produção".
 - Portabilidade: Executa em qualquer ambiente com o runtime de contêiner.
 - Escalabilidade Rápida: Instâncias podem ser provisionadas/destruídas em segundos.
 - Isolamento: Processos isolados uns dos outros.

Orquestração de Contêineres: Kubernetes

- Definição: Plataforma de código aberto para automatizar a implantação, escalonamento e gerenciamento de aplicações containerizadas.
- Criador: Google (baseado em seu sistema Borg).
- Objetivos:
 - Autoescalonamento: Gerencia a elasticidade dos contêineres.
 - Auto-cura: Reinicia contêineres que falham, realoca em nós saudáveis.
 - Balanceamento de Carga: Distribui o tráfego para os contêineres.
 - Descoberta de Serviços: Permite que contêineres se encontrem.
 - Atualizações Rollback: Gerencia atualizações sem tempo de inatividade.
- Componentes Chave: Master (control plane), Nodes (onde os contêineres rodam).

Containers vs. VMs: Comparativo

Característica	Máquinas Virtuais (VMs)	Contêineres
Nível de Abstração	Hardware (Hypervisor)	Sistema Operacional (Engine de Contêiner)
Compartilhamento de Kernel	Não (cada VM tem seu kernel)	Sim (compartilha o kernel do host)
Peso/Tamanho	Mais pesado (SO completo)	Mais leve (apenas app e dependências)
Tempo de Inicialização	Minutos	Segundos/Milissegundos
Isolamento	Forte (hardware-level)	Médio (OS-level, compartilha kernel)
Portabilidade	Alta (imagem da VM)	Extrema (executa em qualquer Docker Engine)
Recursos	Mais consumo (duplica SO)	Menos consumo (compartilha SO)

Serviços Distribuídos: Componentização e Colaboração

- Definição: Arquitetura onde um sistema é dividido em componentes menores, autônomos e independentes que se comunicam através de rede.
- Evolução de: SOA (Service-Oriented Architecture).
- Estilo Mais Popular: Microserviços.
- Objetivos:
 - Desacoplamento: Componentes independentes.
 - Manutenibilidade: Menos código, mais fácil de entender e mudar.
 - Escalabilidade Independente: Cada serviço pode escalar separadamente.
 - Tecnologias Diversas: Cada serviço pode usar a melhor tecnologia para sua função.
- Desafio: Complexidade de gerenciamento, monitoramento, comunicação.

Comunicação entre Serviços Distribuídos (I): REST

- REST (Representational State Transfer):
- Estilo Arquitetural: Não um protocolo, mas um conjunto de princípios para construir APIs web.
- Protocolo Base: HTTP.
- Recursos: Cada URL representa um recurso (ex: /users/123).
- Operações: Usa métodos HTTP (GET, POST, PUT, DELETE) para interagir com recursos.
- Formato de Dados: JSON (mais comum), XML.
- Vantagens: Simples, universal, fácil de usar e depurar.
- Desvantagens: Overhead de HTTP, não otimizado para chamadas de baixa latência.

Comunicação entre Serviços Distribuídos (II): gRPC

- gRPC (gRPC Remote Procedure Call):
- Protocolo: Protocolo RPC (Remote Procedure Call) de código aberto e alto desempenho.
- Tecnologia Base: HTTP/2 (para transporte), Protocol Buffers (para serialização de dados).
- Recursos: Define serviços e métodos (ex: `UserService.GetUser(ID)`).
- Formato de Dados: Protocol Buffers (binário, eficiente).
- Vantagens:
 - Alta Performance: HTTP/2 (multiplexing, streaming), Protocol Buffers (binário).
 - Baixa Latência: Ideal para comunicação interna entre microsserviços.
 - Geração de Código: Suporte a múltiplas linguagens.
- Desvantagens: Mais complexo que REST, menos universal (exige cliente gRPC).

Filas e Mensageria: Desacoplamento e Resiliência

- Definição: Mecanismos de comunicação assíncrona entre componentes de um sistema distribuído.
- Componentes:
 - Produtor: Envia mensagens para a fila.
 - Fila/Tópico: Armazena mensagens temporariamente.
 - Consumidor: Lê e processa mensagens da fila.
- Objetivos:
 - Desacoplamento: Produtor e consumidor não precisam estar online simultaneamente.
 - Resiliência: Mensagens são armazenadas, protegidas contra falhas temporárias.
 - Escalabilidade: Permite que consumidores escalem independentemente.
 - Balanceamento de Carga: Distribui tarefas entre múltiplos consumidores.

RabbitMQ: Message Broker Flexível

- Tipo: Message broker (intermediário de mensagens) de código aberto.
- Protocolo: Implementa o AMQP (Advanced Message Queuing Protocol).
- Modelo: Produtor/Consumidor com trocas (exchanges) e filas.
- Características:
 - Entrega Garantida: Suporte a mensagens persistentes e confirmações.
 - Roteamento Flexível: Diferentes tipos de exchange para roteamento.
 - Sistemas de Filas: Gerencia filas com diferentes políticas.
- Uso Ideal: Cenários que exigem entrega confiável, roteamento complexo e comunicação ponto a ponto ou pub/sub com mensagens individuais.

Apache Kafka: Plataforma de Streaming de Eventos

- Tipo: Plataforma de streaming de eventos distribuída.
- Modelo: Publicação/Inscrição (Publish/Subscribe) em tópicos (topics) com logs de eventos.
- Características:
 - Alto Throughput: Projetado para lidar com milhões de eventos por segundo.
 - Escalabilidade Horizontal: Adiciona brokers para escalar.
 - Persistência: Logs de eventos persistentes (por período configurável).
 - Streaming: Processamento de dados em tempo real.
- Uso Ideal: Processamento de grandes volumes de dados em tempo real, ingestão de dados, construção de pipelines de dados, auditoria, replicação de logs.

BATERIA DE QUESTÕES DE APREDIZAGEM

Ano: 2025 Banca: FUNDEP (Gestão de Concursos) Órgão: UFSJ Prova: FUNDEP (Gestão de Concursos) - 2025 - UFSJ - Analista de Tecnologia da Informação - Desenvolvimento de Sistemas - PROVA LECTOR

1. A arquitetura de microsserviços é um estilo de desenvolvimento de *software* que estrutura uma aplicação como uma coleção de serviços pequenos, autônomos e fracamente acoplados. Essa abordagem contrasta com a arquitetura monolítica, trazendo novas vantagens e desafios.

Sobre a arquitetura distribuída de microsserviços, analise as afirmativas a seguir.

I. Cada microsserviço é desenvolvido, implantado e mantido de forma independente, o que permite que equipes diferentes trabalhem em partes distintas da aplicação sem afetar o todo.

II. Os microsserviços podem ser escritos em diferentes linguagens de programação e utilizar tecnologias de armazenamento distintas, o que permite a escolha da melhor tecnologia para cada funcionalidade.

III. A arquitetura de microsserviços reduz significativamente a complexidade operacional, pois a gestão de um grande número de serviços distribuídos é mais simples do que a de uma única aplicação monolítica.

Estão corretas as afirmativas

- A I e II, apenas.
- B I e III, apenas.
- C II e III, apenas.
- D I, II e III.

Ano: 2025 Banca: Ibest Órgão: CRECI - 11ª Região (SC) Prova: Ibest - 2025 - CRECI - 11ª Região (SC) - Analista de TI

2. Com base nos conceitos de orquestração de contêineres, assinale a alternativa que apresenta a função de uma ferramenta como o Kubernetes.

A Gerencia apenas imagens de contêiner, sem executar serviços de rede, escalonamento ou balanceamento de carga.

B Atua exclusivamente como registrador de logs dos contêineres Docker e não interfere na infraestrutura de rede.

C Permite a execução de scripts Bash automatizados para compilar imagens e enviá-las ao Docker Hub.

D Substitui o Docker Engine, pois executa diretamente os binários dos sistemas operacionais virtualizados.

E É uma plataforma que orquestra contêineres, gerenciando sua criação, escalonamento, atualização, rede e recuperação automática.

Ano: 2025 Banca: CESPE / CEBRASPE Órgão: BANRISUL Prova: CESPE / CEBRASPE - 2025 - BANRISUL - Técnico em Tecnologia da Informação II - Desenvolvimento de Software

3. Em arquiteturas que utilizam mensageria para a comunicação entre sistemas, uma técnica eficiente para a garantia da entrega de mensagens sem perda é

A o processamento imediato de todas as mensagens sem necessidade de recuperação em caso de falha.

B a criação de múltiplos canais simultâneos para envio indiscriminado das mensagens, independentemente da ordem de processamento.

C o armazenamento temporário das mensagens em uma fila persistente até que elas sejam processadas com sucesso.

D o envio repetido de mensagens idênticas sem controle de duplicação para que cheguem ao destino.

E a comunicação direta entre serviços em tempo real, eliminando-se a necessidade de armazenamento intermediário.

Ano: 2025 Banca: CESPE / CEBRASPE Órgão: BANRISUL Prova: CESPE / CEBRASPE - 2025 - BANRISUL - Técnico em Tecnologia da Informação II - Desenvolvimento de Software

4. Em um sistema distribuído que usa eventos para a comunicação entre serviços, uma ação que melhora a escalabilidade da arquitetura é

A a exigência de que cada serviço processador de eventos esteja vinculado a um único emissor, sem possibilidade de assinaturas múltiplas.

B a redução da quantidade de eventos gerados para evitar sobrecarga nos serviços consumidores.

C o envio de eventos de maneira síncrona, para garantir que cada componente processe os dados no mesmo instante em que o evento ocorre.

D a utilização de um barramento de eventos que permita a comunicação assíncrona entre múltiplos serviços interessados na mesma mensagem.

E a imposição de um fluxo fixo e sequencial para cada evento, garantindo-se processamento ordenado sem variações.

Ano: 2022 Banca: IDECAN Órgão: TJ-PI Prova: IDECAN - 2022 - TJ-PI - Analista de Sistemas - Desenvolvimento

5. Application Programming Interface (API) representa um programa que tem por objetivo fornecer ou receber informações de forma facilitada e segura. Ela é amplamente utilizada para a realização de integração entre sistemas, quer seja B2B ou B2C. Especificamente em relação às APIs RESTful, analise as afirmativas abaixo e marque alternativa correta.

I. As APIs RESTful são um tipo de API baseada em tecnologia web.

II. GET, POST, PUT, DELETE e PATCH são exemplos de nomes de verbos das APIs RESTful. Estes determinam a finalidade da requisição que um usuário esteja enviado para a API.

III. Em se tratando de formato de retorno de dados, podemos dizer que as APIs RESTful são agnósticas. Apesar do formato de retorno de dados JSON ser o mais utilizado, outros formatos de retorno podem ser utilizados.

A Apenas as afirmativas I e II estão corretas.

B Apenas as afirmativas I e III estão corretas.

C Apenas a afirmativa III está correta.

D Apenas a afirmativa I está correta.

E Todas as afirmativas estão corretas.

Ano: 2018 Banca: CESPE / CEBRASPE Órgão: Polícia Federal Prova: CESPE - 2018 -
Polícia Federal - Perito Criminal Federal - Área 3

6. Com relação a redes peer-to-peer (P2P), julgue o item subsequente.

O principal objetivo de se usar DHT (distributed hash table) em redes P2P descentralizadas e estruturadas é permitir que cada peer tenha informação total sobre seus vizinhos.

Certo

Errado

Ano: 2025 Banca: UFSM Órgão: UFSM Prova: UFSM - 2025 - UFSM - Professor EBTT
- Área: Ciências Exatas e da Terra/Ciência da Computação/ Metodologia e Técnicas da Computação

7. Os paradigmas de passagem de mensagens e chamada remota a procedimentos (RPC) permitem que componentes de sistemas distribuídos comuniquem-se para a realização de tarefas.

Em relação a esses paradigmas, considere as afirmativas a seguir.

- I → Passagem de mensagens pode ser síncrona ou assíncrona, enquanto RPC é sempre síncrona.
- II → Deadlocks podem ser um problema em sistemas de passagem de mensagens.
- III → RPC requer que cliente e servidor utilizem o mesmo sistema operacional.
- IV → A passagem de mensagens não requer uma arquitetura de memória compartilhada para funcionar corretamente.

Está(ão) correta(s)

A apenas II.

B apenas I e III.

C apenas I e IV.

D apenas II e IV.

E apenas I, III e IV.

Ano: 2023 Banca: VUNESP Órgão: TJ-RS Prova: VUNESP - 2023 - TJ-RS - Analista de Tecnologia da Informação - Especialidade: Análise de Suporte

8. Uma das maneiras de sincronizar sistemas distribuídos é por meio da sincronização de clocks lógicos.

Nesse cenário, um algoritmo que pode ser utilizado para tal propósito é

A a Troca de chaves de Diffie–Hellman.

B o Algoritmo do Avestruz.

C o AES.

D o Algoritmo de Lamport.

E o Round-Robin.

Ano: 2023 Banca: VUNESP Órgão: TJ-RS Prova: VUNESP - 2023 - TJ-RS - Analista de Tecnologia da Informação - Especialidade: Análise de Suporte

9. Durante a implementação de um sistema distribuído com múltiplos nós computacionais, um professor solicitou aos alunos que avaliassem os mecanismos de abstração e coordenação de recursos oferecidos por sistemas operacionais distribuídos. Considerando as propriedades de transparência, escalabilidade e autonomia dos componentes, o recurso que permite ao sistema operacional apresentar ao usuário uma visão unificada e integrada dos recursos distribuídos, mesmo que eles estejam fisicamente espalhados, é denominado:

A distribuição de processos com políticas de escalonamento heterogêneo entre os nós.

B gerenciamento de threads com balanceamento preemptivo entre processos locais.

C segmentação de memória com controle local de páginas físicas por nó.

D transparência de acesso, que oculta ao usuário a localização física dos recursos no sistema distribuído.

E escalonamento assíncrono baseado em relógios lógicos de Lamport.

Ano: 2024 Banca: CESGRANRIO Órgão: IPEA Prova: CESGRANRIO - 2024 - IPEA - Técnico de Planejamento e Pesquisa -Ciência de Dados

10. Para melhorar o processamento de grandes volumes de dados através de computação paralela ou distribuída, pode-se utilizar sharding, técnica que divide os dados em partes menores, chamadas shards. Essas partes são normalmente armazenadas em diferentes nós, ou sítios, de processamento em um sistema distribuído.

O sharding é necessário para garantir a

A privacidade dos dados

B eficiência do processamento de dados

C disponibilidade dos dados em caso de falha de um nó

D escalabilidade do sistema de processamento de dados

E consistência dos dados em caso de alteração de um shard

Ano: 2024 Banca: FGV Órgão: DATAPREV Prova: FGV - 2024 - DATAPREV - ATI - Inteligência da Informação

11. Um dos conceitos associados a coleções de objetos de dados e/ou arquivos de sistemas de bancos de dados NoSQL é o de sharding.

Nesse contexto, uma das características de sharding refere-se

A ao armazenamento de múltiplos itens com timestamps.

B à conciliação de gravação em operações conflitantes.

C a não exigência de um esquema prévio de dados.

D a oferta de uma linguagem de consulta menos poderosa.

E ao particionamento horizontal dos dados entre nós do sistema.

Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CNJ Prova: CESPE / CEBRASPE - 2024 - CNJ - Analista Judiciário - Área Apoio Especializado - Especialidade: Analista de Sistemas

12. Acerca de escalabilidade e tolerância a falhas em sistemas web, julgue o item a seguir.

Escalabilidade vertical ocorre, por exemplo, quando são adicionados mais recursos de CPU e de RAM a um servidor, para aumentar sua capacidade.

Certo

Errado

Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CAU-BR Prova: CESPE / CEBRASPE - 2024 - CAU-BR - Analista de Sistema de Informação

13. Julgue o item subsecutivo, a respeito de integração de dados e big data.

Banco de dados do tipo NoSQL aceitam escalabilidade nas formas horizontal, com o acréscimo de nós para armazenamento, e vertical, com a expansão do processamento dos nós já instalados.

Certo

Errado

Ano: 2025 Banca: FUNDATEC Órgão: UFRGS Prova: FUNDATEC - 2025 - UFRGS -
Analista de TI - Área: Infraestrutura

14. Considerando as características e as responsabilidades do Docker e do Kubernetes, analise as assertivas a seguir:

- I. Docker faz o gerenciamento dos aplicativos encaixotados pelo Kubernetes.
- II. Kubernetes pode ser utilizado com ou sem o Docker.
- III. Docker permite que desenvolvedores de aplicações criem, testem e implantem com flexibilidade e escalabilidade.

Quais estão corretas?

- A Apenas I.
- B Apenas I e II.
- C Apenas I e III.
- D Apenas II e III.
- E I, II e III.

Ano: 2025 Banca: CESPE / CEBRASPE Órgão: SUSEP Prova: CESPE / CEBRASPE - 2025 - SUSEP - Analista Técnico: - Área: Tecnologia da Informação e Ciência de Dados

15. A respeito da arquitetura de sistemas web, julgue o item que se segue.

O gRPC oferece suporte a streaming bidirecional, permitindo que cliente e servidor troquem múltiplas mensagens de forma assíncrona na mesma conexão, por meio da multiplexação do HTTP/2.

Certo

Errado

Ano: 2023 Banca: VUNESP Órgão: TCM-SP Prova: VUNESP - 2023 - TCM-SP - Auxiliar Técnico de Controle Externo - Técnico de Informática

16. No contexto de web services, dentre as características a seguir, aquela que viola uma das restrições impostas pelo estilo arquitetural REST (Representational State Transfer) é:

A o uso de XML na troca de requisições e respostas, no lugar de JSON.

B a possibilidade de clientes ou intermediários na comunicação realizarem cache de respostas, quando estas se definirem como cacheable.

C o uso de outro protocolo, que não o HTTP, para troca de mensagens.

D a possibilidade de um cliente solicitar a exclusão de um recurso no servidor.

E o armazenamento no servidor, entre requisições, de informações de contexto do cliente ou estado de sessão.

Ano: 2024 Banca: IF-TO Órgão: IF-TO Prova: IF-TO - 2024 - IF-TO - Analista de Tecnologia da Informação

17. A sigla REST ou REpresentational State Transfer (ou Transferência de Estado Representativo, em tradução livre), muito utilizadas nos contextos atuais de desenvolvimento de software. Sobre ela responda.

A Surgiu como um conceito novo, mas que não gerou resultados ou melhorias nas interconexões entre diferentes bancos de dados.

B É um framework independente de linguagem de programação usado em desenvolvimento de software focado em camadas de negócio e proximidades com os usuários, facilita a comunicação entre sistemas na internet.

C É uma linguagem de alto nível para integração de sistemas na web.

D É um banco de dados para grande quantidade de informações, principalmente baseadas no armazenamento em nuvem.

E É uma forma de realizar a comunicação e a troca de dados entre dois sistemas diferentes, independentemente das linguagens e tecnologias utilizadas no desenvolvimento. Ele usa protocolos padrões, como HTTP, e pode transmitir dados usando formatos como XML e JSON.

Ano: 2024 Banca: IBFC Órgão: TRF - 5ª REGIÃO Prova: IBFC - 2024 - TRF - 5ª REGIÃO - Técnico Judiciário - Área de Apoio Especializado - Especialidade Desenvolvimento de Sistemas da Informação

18. Sobre RabbitMQ, analise as afirmativas abaixo e dê valores Verdadeiro (V) ou Falso (F).

- ☐ RabbitMQ é um middleware de mensageria open-source que implementa o protocolo Advanced Message Queuing Protocol (AMQP).
- ☐ O RabbitMQ é exclusivo para a linguagem de programação Java, não oferecendo suporte a outras linguagens de programação.
- ☐ No RabbitMQ, os produtores são responsáveis por enviar mensagens para uma fila, enquanto os consumidores as recebem e processam.

Assinale a alternativa que apresenta a sequência correta de cima para baixo.

Alternativas

A F - F - F

B F - V - F

C V - F - V

D V - V - V

Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CAU-BR Prova: CESPE / CEBRASPE - 2024 - CAU-BR - Analista de Sistema de Informação

19. Acerca do Confluent Kafka, julgue o item a seguir.

Uma aplicação que publica dados escrevendo em um tópico no Kafka cluster é considerada producer.

Certo

Errado

Ano: 2024 Banca: UFMT Órgão: Prefeitura de Cáceres - MT Prova: UFMT - 2024 - Prefeitura de Cáceres - MT - Analista de Sistemas

20. Ao considerar a arquitetura de sistemas de mensageria, qual afirmativa apresenta a diferença entre Apache Kafka e RabbitMQ?

A Ambos Kafka e RabbitMQ utilizam o modelo de filas, mas Kafka prioriza a entrega garantida e a ordem das mensagens, enquanto RabbitMQ enfatiza a escalabilidade horizontal.

B RabbitMQ é mais adequado para ambientes de alta latência, em que a consistência imediata é crucial, enquanto Kafka é projetado para otimizar a entrega assíncrona em grande escala.

C Kafka e RabbitMQ são intercambiáveis em termos de modelo de mensagens, e a escolha entre eles é baseada apenas na preferência do desenvolvedor em relação à linguagem de programação.

D Kafka segue um modelo de log de eventos distribuído, permitindo a retenção de mensagens por um período específico, enquanto RabbitMQ utiliza uma fila de mensagens tradicional.